☁ 영남대학교 Agentic AI 강의 과정

Claude Code 심화와 스킬 사용 준비 (3주차 - 5일차 이론)

대상: Claude를 처음 쓰는 입문자 학생

일정: 2026. 11. 14(토) 13:00 ~ 18:00

강사: 신인철 (클라우드스퀘어)

> ℹ️ **오늘의 한 줄 목표**
> "CLAUDE.md를 **짧고 정확하게 키우고, 나누고, 문서 스킬(Word · Excel · PPT · PDF)** 을 설치해 바로 써 보며, AI를 **내 방식대로 일하는 동료**로 만든다!"

🕐 5일차 — CLAUDE.md 심화와 스킬 사용 준비

2026. 11. 14(토) 13:00 ~ 18:00 시간표

이론 3시간 · 실습 1.5시간 · 정리 30분 · 50분 수업 + 10분 휴식 · 토 · 일 13:00 ~ 18:00 · 총 8일 / 40시간

시간	구분	내용
13:00 ~ 13:50	이론 1	4일차 회고 · CLAUDE.md가 읽히는 방식 (위치 계층 · 우선순위 · 컨텍스트 비용)
13:50 ~ 14:00	휴식	🍵 10분 휴식
14:00 ~ 14:50	이론 2	잘 쓰는 법 — 짧게 쓰기 · `@` 가져오기 · 규칙 폴더 · 교정 저장 습관 · `/clear` 와의 관계
14:50 ~ 15:00	휴식	🍵 10분 휴식
15:00 ~ 15:50	이론 3	역할 나누기(CLAUDE.md vs 스킬 vs Hooks) · 문서 스킬 설치 · 다른 에이전트(OpenClaw · Hermes) · Git
15:50 ~ 16:00	휴식	🍵 10분 휴식
16:00 ~ 16:50	실습 ①	주제 선택 · 프로그램 설치 → CLAUDE.md 없이 vs 있을 때 비교 → `@` 가져오기 · 교정 저장 (Lab 00~04)
16:50 ~ 17:00	휴식	🍵 10분 휴식
17:00 ~ 17:30	실습 ②	문서 스킬 설치 → PDF 만들고 읽어서 PNG · Excel · Word · PPT로 바꾸기 (Lab 05~06)
17:30 ~ 18:00	정리	배포 · 비교 · 서버 이미지 생성 · 점검표 · FAQ · Day 6 예고 (Lab 07)

🗺️ 8일간의 Agentic AI 마스터 로드맵 (3~4주차)

기억시키기(Day 5) → 일 시키기(Day 6) → 팀 프로젝트(Day 7~8)

Day 5 · 11/14(토) · 오늘
CLAUDE.md 심화 · 스킬 준비

무엇을 기억시킬까

위치 계층 · 짧게 쓰기 · `@` 가져오기 · 교정 저장 · 문서 스킬 설치 · OpenClaw · Hermes 비교

Day 6 · 11/15(일)
스킬 본격 활용 · 확장

어떻게 일을 시킬까

스킬 잘 쓰기 · 나만의 스킬 · Plan 모드 · 서브에이전트 · Hooks · MCP · 프로젝트 아이디어

Day 7 · 11/21(토)
팀 프로젝트

팀으로 만들기

조 편성 · 주제 · 팀 CLAUDE.md · 대표 서버 배포 · 발표 자료

Day 8 · 11/22(일)
Demo Day · 수료

보여주기

팀 발표 · 라이브 시연 · 종합 퀴즈 · 서버 정리

🎯 Day 5에서 완성할 결과물

📁 /root/day5-lab/ — 오늘 실습 폴더

CLAUDE.md
짧고 정확한 프로젝트 규칙 (직접 키운 버전)

.claude/rules/ · docs/
`@` 가져오기로 나눈 세부 규칙

📦 문서 스킬 (플러그인)
Word · Excel · PPT · PDF + 필요한 프로그램

Claude Code
→
생성 · 배포

🌐 /var/www/html/day5/

주제별 웹페이지
동아리 소개 · 학습 도우미 · 포트폴리오 중 택 1

📄 /day5/files/
내가 만든 문서 파일 · 다운로드 링크

구분	내용
필수	프로그램 설치 → CLAUDE.md 없이 vs 있을 때 비교 → 규칙 나누기 → 교정 저장 → **문서 스킬 설치** → **PDF 만들고 1종 이상으로 바꾸기** → 서버 이미지
선택	문서 2~4종 · LibreOffice 설치 · GitHub에 커밋

Day 5 이론 학습 목차

Part	주제	핵심 질문
1	Day 4 회고 · 오늘의 질문	기초에서 무엇이 더 필요할까?
2	CLAUDE.md가 읽히는 방식	어떤 파일이, 언제, 어떤 순서로 읽히나?
3	잘 쓰는 법 ① 짧고 정확하게	무엇을 넣고 무엇을 빼야 하나?
4	잘 쓰는 법 ② 나누기 — `@` 가져오기 · 규칙 폴더	길어진 규칙은 어떻게 정리하나?
5	잘 키우는 법 — 반복 개선	규칙은 언제, 어떻게 추가하나?
6	역할 나누기 — CLAUDE.md vs 스킬 vs Hooks	어떤 내용을 어디에 두나?
7	스킬 사용 준비 — 문서 스킬 설치와 기본 사용	스킬은 어디서 오고, 어떻게 설치하고 부르나?
8	다른 AI 에이전트 — OpenClaw · Hermes	Claude Code와 무엇이 다른가?
9	Git으로 공유 & 실습 미리보기	무엇을 올리고 무엇을 빼나?

🔁 Part 1. Day 4 회고 · 오늘의 질문

CLAUDE.md를 "만들 줄 안다"에서 "잘 쓴다"로

1.1 Day 4 회고 — 서버 위의 AI 에이전트

Day 4 ①
Claude Code

설치 · 로그인
서버에 설치 · 팀 계정 로그인

에이전트
요청 → 계획 → 실행 → 확인 → 보고

Day 4 ②
CLAUDE.md 기초

개념
시작할 때마다 자동으로 읽는 지침서

만들기
직접 쓰기 · `/init` · `/memory`

Day 4 ③
안전

여러 겹의 안전장치
규칙 · 승인 읽기 · 백업 · 서버 이미지

Day 4 결과물
메모 앱

v1~v4
달력 연동 · Flask API · Nginx 연결

과제
v5~v6 · GitHub

💡 **오늘의 질문**: 규칙이 **10줄일 때**는 잘 지켰는데, **100줄이 되면**? 같은 말을 **세 번째** 고쳐 주고 있다면? 매번 같은 요청 문장을 **복사해서 붙여넣고** 있다면? → 오늘 배울 **심화 CLAUDE.md** 와 **스킬**

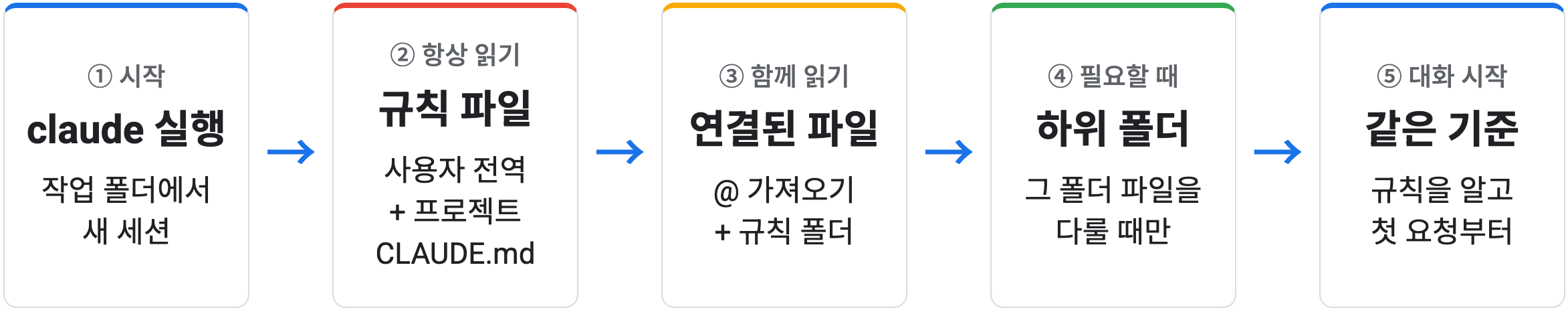
1.2 Day 4 기초 vs Day 5 심화

구분	Day 4 기초 (이미 배움)	Day 5 심화 (오늘)
개념	자동으로 읽히는 지침서	매 대화 **컨텍스트를 차지** → 짧게
위치	사용자 · 프로젝트 · 하위 폴더	위치별 **역할 설계** · 개인용 · 규칙 폴더
작성	직접 · `/init` · `/memory`	없이 시작해 **관찰** → `/init` → 다듬기
관리	기본 구조 4가지	`@` **가져오기**로 나누기
습관	—	교정 내용을 **CLAUDE.md에 저장**
한계	지침서이지 잠금장치가 아님	꼭 실행할 일은 **Hooks** (Day 6)
확장	—	필요할 때만 읽는 **스킬**

📚 Part 2. CLAUDE.md가 읽히는 방식

어떤 파일이 · 언제 · 어떻게 읽히고 · 무엇을 차지하나

2.1 세션을 시작하면 일어나는 일



- ②③은 **대화마다 처음부터 책상 위에 펼쳐져 있는 문서**입니다. → 길면 길수록 자리를 차지 (2.4)
- ④는 Claude가 그 폴더의 파일을 **읽거나 고칠 때 추가로** 읽힙니다.
- 작업 폴더의 **상위 폴더**에 있는 CLAUDE.md도 함께 읽힙니다. (예: `/root/CLAUDE.md`)

2.2 위치별 역할 — 5가지 자리

구분	위치	언제 읽히나	무엇을 적나	Git
사용자 전역	`~/.claude/CLAUDE.md`	항상 (모든 프로젝트)	내 습관 · 답변 언어 · 말투	✕
프로젝트	`./CLAUDE.md` 또는 `./.claude/CLAUDE.md`	항상 (이 폴더)	프로젝트 개요 · 명령 · 규칙	✅
개인 프로젝트	`./CLAUDE.local.md`	항상 (이 폴더 · 나만)	내 서버 IP · 개인 메모	✕
규칙 폴더	`./.claude/rules/*.md`	항상 또는 조건부	주제별로 나눈 세부 규칙	✅
하위 폴더	`./web/CLAUDE.md` 등	그 폴더를 다룰 때	그 부분만의 규칙	✅

💡 **설계 원칙**: "**나에게만** 해당하나?" → 사용자 전역 / 개인 프로젝트 · "**이 프로젝트 모두에게** 해당하나?" → 프로젝트 / 규칙 폴더

2.2 위치별 역할 — 5가지 자리 (확대 보기)

👤 ~/.claude/CLAUDE.md — 나의 모든 프로젝트 공통 (Git ✕)

📁 /root/day5-lab/CLAUDE.md — 프로젝트 공통 규칙 (Git ✅)

🔒 CLAUDE.local.md
나만 쓰는 메모 (Git ✕)

📂 .claude/rules/*.md
주제별 세부 규칙 (Git ✅)

📁 web/CLAUDE.md
그 폴더를 다룰 때만 (Git ✅)

2.3 여러 파일이 겹치면? — 우선순위와 충돌

- 모든 파일은 **덮어쓰지 않고 함께** 읽힙니다.
- 보통 **더 구체적인(안쪽) 파일**이 세부 사항을 보완합니다.
  - 전역: "답변은 한국어로"
  - 프로젝트: "코드 주석은 영어로"
  - → 두 규칙 모두 적용
- **서로 반대되는 규칙**이 있으면 Claude도 헷갈립니다. 어느 쪽을 따를지 **보장되지 않습니다.**

😵 충돌 예시	✅ 고친 예시
전역: "들여쓰기 4칸"	전역: (삭제)
프로젝트: "들여쓰기 2칸"	프로젝트: "들여쓰기 2칸"

⚠️ **규칙 충돌 점검**: 새 규칙을 추가할 때 `/memory` 로 **이미 읽히는 파일 목록**을 열어 보고 **같은 주제의 규칙**이 있는지 먼저 확인합니다.

2.4 컨텍스트 창 — AI의 "작업 책상"

🗃️ 컨텍스트 창 = 한 번에 펼쳐 놓을 수 있는 책상 크기 (한정됨)

📘 CLAUDE.md	💬 대화 내용	📄 읽은 파일	🧰 도구 결과
항상 펼쳐져 있음	주고받을수록 쌓임	코드 · 결과	명령 실행 출력

책상이 가득 차면	그래서
오래된 대화가 **요약 · 정리**되고 세부 내용이 흐려짐	CLAUDE.md는 **짧게** — 매 대화 자리를 차지
규칙이 많을수록 **중요한 규칙이 묻힘**	꼭 필요한 규칙만, 나머지는 **필요할 때 읽게** (스킬 · 하위 폴더)
응답이 느려지고 비용(사용량)이 늘어남	`/context` 로 **무엇이 얼마나 차지하는지** 확인

2.5 지금 무엇이 읽혔나? — `/memory` 와 `/context`

명령 1
`/memory`

읽힌 파일 목록
지금 세션에 적용된 CLAUDE.md · 규칙 파일을 확인

바로 편집
목록에서 골라 편집기로 열기

명령 2
`/context`

책상 사용량
컨텍스트 창을 무엇이 얼마나 차지하는지 표시

점검 포인트
메모리 파일(CLAUDE.md) 비중이 너무 크지 않은가?

💡 **습관**: CLAUDE.md를 고친 뒤에는 `/memory` **로 읽혔는지**, 규칙을 많이 늘린 뒤에는 `/context` **로 크기**를 확인합니다. (실습 Lab 02 · Lab 03)

✂️ Part 3. 잘 쓰는 법 ① 짧고 정확하게

"이 줄을 지우면 Claude가 실수할까?"

3.1 좋은 규칙 vs 나쁜 규칙

😵 나쁜 규칙 (모호함)	✅ 좋은 규칙 (구체적 · 확인 가능)
코드를 깔끔하게 작성한다	들여쓰기는 **2칸**, 함수 위에 **한 줄 한국어 주석**
예쁘게 디자인한다	색은 `style.css` 의 **변수 4개만** 사용 (`--main` , `--sub` …)
꼼꼼하게 확인한다	수정 후 **`curl -I http://localhost/day5/`** 로 200 확인
중요한 파일은 조심한다	**`index_day1.html` 수정 · 삭제 금지**
답변을 잘 정리한다	끝나면 **변경한 파일 목록**을 표로 보여준다

💡 **기준**: 사람이 봐도 **"지켰다 / 안 지켰다"를 판단할 수 있는 문장**인가? 판단할 수 없다면 Claude도 지키기 어렵습니다.

3.2 넣을 것 vs 뺄 것

✅ 넣을 것

Claude가 추측할 수 없는 것

· 실행 · 확인 방법 (명령어 그대로)
· 우리만의 규칙 (파일 이름 · 폴더 구조 · 스타일)
· 금지 사항 · 안전 규칙
· 자주 틀리는 부분과 올바른 방법

✕ 뺄 것

보면 알거나 자주 바뀌는 것

· 코드를 읽으면 알 수 있는 설명
· 일반 상식 ("HTML은 웹 언어다")
· 자주 바뀌는 정보 (오늘 할 일 목록)
· 비밀번호 · API 키 · 개인 정보

❗ **한 줄 점검 질문**: "이 줄을 지우면 Claude가 **실수할까?**" → 아니라면 지웁니다. 짧을수록 **남은 규칙이 더 잘 지켜집니다.**

3.3 강조 표현은 아껴 쓰기

- "**중요**", "**반드시**", "**절대**" 같은 강조는 Claude가 더 주의하게 만듭니다.
- 하지만 **모든 줄을 강조**하면 아무것도 강조되지 않습니다.
- 강조는 **어기면 되돌리기 어려운 규칙**에만 사용합니다.
  - 파일 삭제 · 서버 설정 · 비밀 정보
- 형식은 **목록(`-`) + 짧은 문장**이 가장 잘 지켜집니다.

```
## 작업 규칙
- 들여쓰기 2칸
- 새 페이지는 pages/ 폴더에 만든다
- 수정 후 변경 파일 목록을 표로 보여준다

## 금지 (반드시 지킬 것)
- index_day1.html 수정 · 삭제 금지
- rm -rf 사용 금지
```

🧩 Part 4. 잘 쓰는 법 ② 나누기

@ 가져오기 · 규칙 폴더 · 개인용 파일

4.1 @ 가져오기 — 다른 파일을 연결하기

```
# 동아리 소개 사이트

## 개요
- 영남대 OO 동아리 소개 웹사이트
- 자세한 소개 문구는 @docs/club-info.md

## 디자인 규칙
@docs/design.md

## 작업 규칙
- 새 페이지는 pages/ 폴더에 만든다
```

- CLAUDE.md 안에 `@파일경로` 를 쓰면 그 파일 내용도 **함께 읽힙니다.**
- 경로는 **CLAUDE.md 기준 상대 경로** (`@docs/design.md`) 또는 **절대 경로** (`@~/notes/my.md`)
- 가져온 파일 안에서 **또** `@` 로 다른 파일을 가져올 수 있습니다. (단계 수 제한 있음)
- 코드 블록(```) 안의 `@` 는 가져오기로 처리되지 않습니다.

⚠️ **주의**: 가져온 파일도 **항상 읽힙니다.** 나눈다고 짧아지는 것이 아니라 **정리가 잘 되는 것**입니다. 가끔만 필요한 내용은 **스킬** (Part 7)

4.1 @ 가져오기 — 예시 파일 (확대 보기)

```
# 동아리 소개 사이트

## 개요
- 영남대 OO 동아리 소개 웹사이트
- 자세한 소개 문구는 @docs/club-info.md

## 디자인 규칙
@docs/design.md

## 작업 규칙
- 새 페이지는 pages/ 폴더에 만든다
```

4.2 왜 나누나? — 한 파일 vs 나눈 파일

😵 **한 파일에 전부 (120줄)**

개요 · 명령 · 디자인 규칙 · 글쓰기 규칙 · 금지 사항 · 동아리 소개 문구가 한데 섞임

→ 어디를 고쳐야 할지 찾기 어려움
→ 디자인 담당 · 글 담당이 같은 파일을 동시에 수정

정리 →

✅ **나눈 구조**

`CLAUDE.md` (25줄) · 개요 · 명령 · 금지
`docs/design.md` · 색 · 글꼴 · 레이아웃
`docs/writing.md` · 말투 · 맞춤법
`docs/club-info.md` · 소개 문구

→ 담당별로 **한 파일만** 고치면 됨

- 이미 있는 문서(`README.md` 등)를 **다시 쓰지 않고** `@README.md` 로 연결할 수 있습니다.
- 나눈 파일은 **사람에게도 읽기 좋은 문서**가 됩니다. (팀 과제 · 팀 프로젝트)

4.3 규칙 폴더 `.claude/rules/` — 주제별 규칙 파일

```
day5-lab/
├── CLAUDE.md
└── .claude/
    └── rules/
        ├── design.md      ← 항상 읽힘
        ├── writing.md     ← 항상 읽힘
        └── html.md        ← HTML 다룰 때만
```

```
---
paths:
  - "**/*.html"
---
# HTML 규칙
- <head>에 style.css 를 연결한다
- 이미지에는 alt 를 반드시 쓴다
```

- `.claude/rules/` 안의 `.md` 파일은 `@` **없이도 자동으로** 읽힙니다.
- 파일 맨 위에 `paths` 를 쓰면 **해당 파일을 다룰 때만** 읽힙니다. (조건부 규칙)
  - `"**/*.html"` → HTML 파일 작업 시
  - `"pages/**"` → pages 폴더 작업 시
- **항상 필요한 규칙**은 CLAUDE.md, **특정 파일에만 필요한 규칙**은 `paths` 규칙 파일

4.3 규칙 폴더 `.claude/rules/` — 조건부 규칙 예시 (확대 보기)

```
---
paths:
  - "**/*.html"
---
# HTML 규칙
- <head>에 style.css 를 연결한다
- 이미지에는 alt 를 반드시 쓴다
```

paths	언제 읽히나
(없음)	**항상** — CLAUDE.md처럼 세션 시작 시
`"**/*.html"`	HTML 파일을 읽거나 고칠 때
`"pages/**"`	`pages/` 폴더 안의 파일을 다룰 때

4.4 나만의 설정은 `CLAUDE.local.md`

파일	누구 것?	Git에 올리나?	예시
`CLAUDE.md`	**팀 · 프로젝트 공통**	✅	폴더 구조 · 디자인 규칙 · 금지 사항
`CLAUDE.local.md`	**나만** (이 프로젝트)	✕	내 서버 공인 IP · 내 작업 메모 · 내 실습 진도
`~/.claude/CLAUDE.md`	**나만** (모든 프로젝트)	✕	"한국어로 답변" · "명령 전 한 줄 설명"

```
echo "CLAUDE.local.md" >> .gitignore
```

🚫 **그래도 비밀번호 · API 키는 어디에도 쓰지 않습니다.** CLAUDE.local.md도 서버에 남는 일반 텍스트 파일입니다.

🌱 Part 5. 잘 키우는 법 — 반복 개선

처음부터 완벽하게 쓰지 않는다 — 관찰하고, 고치고, 저장한다

5.1 CLAUDE.md를 키우는 순서

① 없이 시작
관찰
CLAUDE.md 없이 작은 작업을 맡겨 봄

→

② 초안
`/init`
파일이 생기면 초안 자동 생성

→

③ 다듬기
사람이 검토
모호한 문장 삭제
금지 사항 추가

→

④ 교정 저장
실수 → 규칙
같은 실수를 하면 한 줄 추가

→

⑤ 정리
줄이기
안 쓰는 규칙 삭제
길면 나누기

- **왜 없이 시작하나?** 처음부터 상상으로 쓴 규칙은 **실제로 필요 없는 규칙**이 많습니다. Claude가 **무엇을 잘하고 무엇을 틀리는지** 먼저 보면 꼭 필요한 규칙만 남습니다.
- **실습 Lab 01 · 02**: CLAUDE.md **없이** 페이지를 만들고 → 규칙을 쓴 뒤 **같은 요청**으로 다시 만들어 비교합니다.

5.2 교정 내용을 저장하는 습관

👦 **학생** "새 페이지 만들 때 또 `style.css` 를 연결 안 했네요. 고쳐 주세요."

▼

🤖 **Claude** " `pages/members.html` 의 <head>에 style.css 연결을 추가했습니다."

▼ 여기서 한 번 더!

👦 **학생 "방금 고친 내용을 다시 실수하지 않도록 CLAUDE.md 작업 규칙에 한 줄로 추가해줘."**

▼

🤖 **Claude** CLAUDE.md에 `- 새 HTML 페이지는 <head>에 style.css 를 연결한다` 추가 → **사람이 승인**

💡 **기준**: 같은 지적을 **두 번째** 하게 되면 → CLAUDE.md에 저장할 때입니다. 저장한 뒤에는 **새 세션**에서 같은 요청으로 확인합니다.

5.3 `/clear` · `/compact` 와 CLAUDE.md

명령	하는 일	대화 내용	CLAUDE.md
`/clear`	대화를 **완전히 비우고** 새로 시작	사라짐	**다시 읽힘** ✅
`/compact`	지금까지의 대화를 **요약**해 자리 확보	요약만 남음	**유지** ✅
`/exit` 후 다시 `claude`	새 세션	사라짐	**다시 읽힘** ✅

- 주제가 바뀌면 `/clear` 로 책상을 비우는 것이 좋습니다. (이전 대화가 새 작업을 방해하지 않도록)
- 대화 중에 정한 중요한 결정은 **사라지기 전에 CLAUDE.md로 옮겨야** 다음에도 남습니다.

❗ **질문 하나로 판단**: "**다음 세션에서도** 이 내용이 필요할까?" → 예: CLAUDE.md에 저장 · 아니오: 대화로 충분

5.4 정기 점검 — 내 CLAUDE.md 점검표

점검 1
짧은가?

· 한 줄 한 규칙인가?
· "지우면 실수할까?"에 아닌 줄은 없나?
· 너무 길면 `@` · 규칙 폴더로 나눴나?

점검 2
정확한가?

· 명령어는 실제로 동작하나?
· 폴더 · 파일 이름이 지금과 같나?
· 서로 반대되는 규칙은 없나?

점검 3
안전한가?

· 비밀 정보가 들어 있지 않나?
· 금지 사항이 빠지지 않았나?
· Git에 올릴 파일과 뺄 파일을 구분했나?

💡 **Claude에게 점검을 맡겨도 됩니다**: "CLAUDE.md를 읽고 **모호하거나 중복되거나 서로 충돌하는 규칙**을 찾아서 표로 보여줘. 고치지는 말고." → 사람이 보고 결정

⚖️ Part 6. 역할 나누기 — CLAUDE.md vs 스킬 vs Hooks

항상 기억할 것 · 필요할 때 꺼낼 것 · 반드시 실행할 것

6.1 세 가지 도구 비교

📘 CLAUDE.md
항상 기억할 것

읽히는 때
매 세션 시작 · 항상

성격
지침 (따르도록 **안내**)

예시
폴더 구조 · 금지 사항 · 답변 언어

🧰 스킬 (Skill)
필요할 때 꺼낼 것

읽히는 때
평소엔 **이름 · 설명만** → 필요할 때 전체

성격
특정 작업의 **업무 매뉴얼**

예시
보고서 서식 · 이메일 감수 절차

⚙️ Hooks
반드시 실행할 것

실행되는 때
정해진 순간 (파일 수정 후 등)

성격
AI 판단과 무관하게 **자동 실행**

예시
수정 후 항상 문법 검사 · 백업

ℹ️ **Day 3 Cowork와 비교**: Cowork의 **지침**은 CLAUDE.md, Cowork의 **스킬**은 Claude Code의 스킬과 같은 아이디어입니다. Hooks는 **Day 6**에서 직접 실습합니다.

6.2 이 내용은 어디에? — 판단 연습

내용	어디에?	이유
"답변은 항상 한국어로"	🟦 사용자 전역 CLAUDE.md	모든 프로젝트 · 항상
"새 페이지는 `pages/` 에 만든다"	🟦 프로젝트 CLAUDE.md	이 프로젝트 · 항상
"HTML의 이미지에는 alt 필수"	📂 `.claude/rules/` (`paths`)	HTML 작업 때만
"보고서는 표지 · 목차 · 요약 순서로"	🧰 스킬	보고서 쓸 때만 · 절차가 김
"교수님께 보내는 이메일 감수 절차"	🧰 스킬	가끔 · 단계별 절차
"HTML 수정 후 **반드시** 문법 검사"	⚙️ Hooks	한 번도 빠지면 안 됨

💡 **세 가지 질문**: ① 매번 필요한가? → CLAUDE.md ② 특정 작업에만 필요하고 길다? → 스킬 ③ 한 번도 빠지면 안 되나? → Hooks

6.3 "지시"와 "보장"의 차이 — Hooks 맛보기

📘 CLAUDE.md에 쓴 경우 — 지시

`- HTML을 수정하면 검사 명령을 실행한다`

→ 대부분 지키지만, 대화가 길어지거나 급한 요청이면 **빠질 수 있음**

차이 ⇄

⚙️ Hooks로 설정한 경우 — 보장

`.claude/settings.json` 에
"파일 수정 후 → 검사 명령 실행" 등록

→ Claude가 잊어도 **프로그램이 항상 실행**

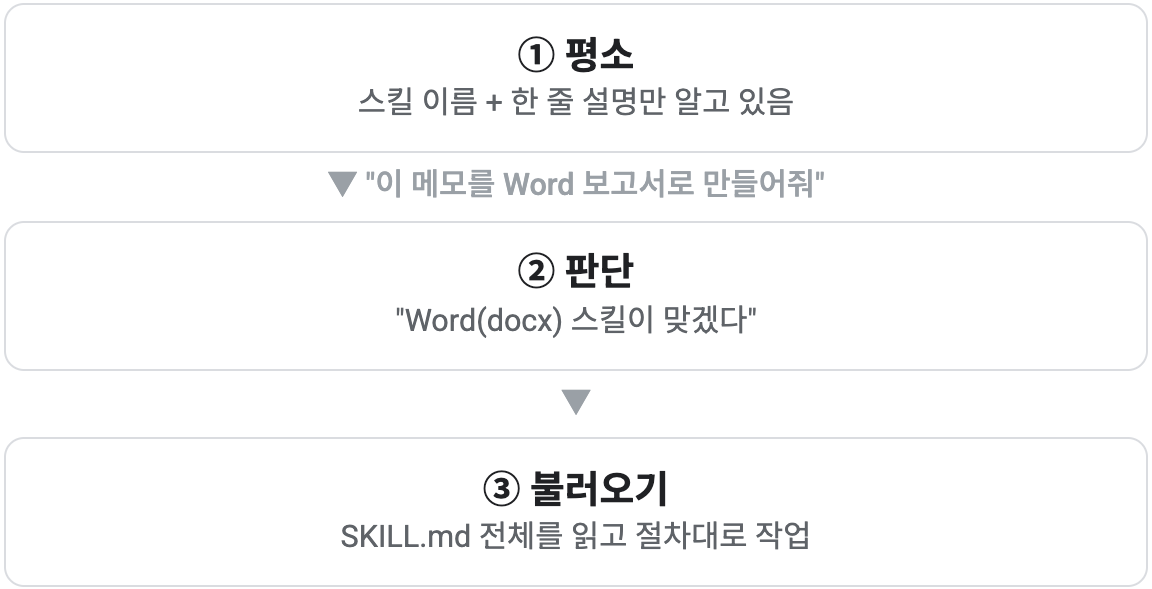
- CLAUDE.md는 **AI가 읽고 따르는 글**, Hooks는 **Claude Code 프로그램이 지키는 설정**입니다.
- 오늘은 **개념만** 알아 두고, **Day 6**에서 "수정 후 자동 검사" Hook을 직접 만들어 봅니다.

🧰 Part 7. 스킬 사용 준비 — 문서 스킬 설치와 기본 사용

스킬은 어디서 오고, 무엇이 필요하고, 어떻게 설치하고 부르나

7.1 스킬이란? — 필요할 때 꺼내 쓰는 업무 매뉴얼

- **스킬(Skill)** = 특정 작업을 **어떤 순서와 기준으로 할지** 적은 폴더
  - 핵심 파일은 `SKILL.md` 하나 + 필요한 도구 · 양식 파일
- 평소에는 **이름과 설명(description)만** 알고 있다가, 요청이 그 설명과 맞으면 **그때 전체를 읽습니다.**
- 그래서 스킬은 **많이 설치해도 책상을 거의 차지하지 않습니다.**



ℹ️ **Day 3 Cowork와 연결**: Cowork에서 Word · PPT 파일을 만들 때 쓰던 것이 바로 **문서 스킬**입니다. 오늘은 같은 계열의 스킬을 **서버의 Claude Code에 직접 설치**합니다.

7.2 스킬은 어디서 오나? — 3가지 출처

출처 1
기본 내장

설치 없음

Claude Code에 처음부터 들어 있는 스킬 · 명령

출처 2 · 오늘
플러그인

묶음으로 설치

공식 · 다른 사람이 만든 스킬 묶음을 명령 한 번으로 설치

부르기

`/플러그인:스킬`

예) `/document-skills:docx`

출처 3 · Day 6
내 폴더

직접 만들기

`~/.claude/skills/` (나) · `.claude/skills/` (프로젝트)

부르기

`/스킬이름`

- `/skills` 를 입력하면 세 출처의 스킬이 **모두 목록으로** 보입니다.
- 오늘은 **출처 2(플러그인)** 로 문서 스킬을 설치해 쓰고, **출처 3(직접 만들기)** 은 **Day 6**에서 합니다.

7.3 문서 스킬 4종 — 오늘은 "PDF를 읽고 바꾸기"

스킬	하는 일	오늘 실습 (PDF 입력 →)	결과 확인
pdf	PDF 만들기 · 읽기 · 표 추출 · 페이지 → 이미지 · 합치기	보고서 PDF 만들기 · **PNG 이미지** · 워터마크	브라우저
xlsx (Excel)	표 · 수식 · 차트	PDF의 표 → **엑셀 표 + 합계 + 차트**	Excel · 구글 시트
docx (Word)	보고서 · 회의록 · 이력서	PDF 요약 → **1쪽 요약 보고서**	Word · 구글 문서
pptx (PowerPoint)	발표자료	PDF 핵심 → **3장 발표자료**	PowerPoint · 구글 슬라이드

- 네 가지 모두 공식 스킬 묶음 `document-skills` 에 들어 있습니다. **PDF는 브라우저 · 폰에서 바로** 열리고, Office 파일은 **구글 드라이브**로도 열 수 있습니다.

> ℹ **Claude는 PDF를 직접 읽을 수 있는데 왜 스킬이?** — 요약은 스킬 없이도 됩니다. 스킬은 **파일을 만들고 · 표를 정확히 추출하고 · 결과를 검사하는 절차**를 더해 줍니다.

7.4 스킬만으로는 부족하다 — 필요한 프로그램 설치

📘 스킬 (SKILL.md)
절차 · 기준을 알려줌
"docx 라이브러리로 만들고, 검사하고…"

+

🛠️ 서버 프로그램
실제로 파일을 만드는 도구
Node.js · Python 라이브러리 · 변환 도구

=

✅ 결과 파일
.docx · .xlsx
.pptx · .pdf

구분	설치할 것	쓰는 스킬
Node.js 22 + npm 패키지	`docx` · `pptxgenjs`	Word · PPT
Python 라이브러리	`openpyxl` · `pandas` · `reportlab` · `pypdf` · `pdfplumber` · `markitdown` · `matplotlib`	Excel · PDF · 읽기 · 차트
도구 · 글꼴	`poppler-utils` · `pandoc` · `qpdf` · **나눔 글꼴**	확인 · 변환 · 한글 PDF
(선택) **LibreOffice**	화면 없이 쓰는 변환 프로그램	Excel 수식 재계산 · PDF 변환

💡 스킬은 Anthropic 작업 환경에 프로그램이 **이미 있다고 가정**하고 쓰여 있습니다. 우리 서버에는 **실습 Lab 00에서 미리 설치**합니다.

7.5 설치 방법 ① 플러그인으로 설치 (기본)

① 저장소 등록

marketplace add

공식 스킬 저장소를 플러그인 목록에 추가

→

② 설치

plugin install

`document-skills`

묶음 설치

→

③ 적용

reload

`/reload-plugins`

→

④ 확인

`/skills`

docx · xlsx · pptx · pdf

```
/plugin marketplace add anthropics/skills
/plugin install document-skills@anthropic-agent-skills
```

- 명령 대신 `/plugin` 화면에서 저장소(`anthropic-agent-skills`) → `document-skills` → **설치** 순서로 골라도 됩니다.

7.6 설치 방법 ② 플러그인이 막혀 있을 때 (대안)

왜 막힐 수 있나?

- 팀 계정 **관리자 설정**에서 플러그인 설치를 제한
- 서버에서 GitHub 접속이 안 되는 네트워크

대안 A — 스킬 없이 직접 요청

- Lab 00에서 설치한 라이브러리로 Claude Code에게 **"Word 파일로 만들어줘"** 라고 직접 요청 → 파일은 만들어짐
- 스킬이 있을 때와 **결과 · 과정의 차이**를 비교하는 실습이 됨

대안 B — 공개 스킬을 폴더로 복사

- 공식 저장소의 **Apache 2.0 라이선스 스킬** (예: `theme-factory` 문서 · 발표 테마)을 `~/.claude/skills/` 에 복사 → `/reload-skills`
- **폴더로 설치하는 방법**을 익혀 두면 Day 6 나만의 스킬로 이어짐

> 🚫 **문서 스킬 4종(docx · xlsx · pptx · pdf)은 폴더로 복사하지 않습니다.** 공개만 된 **독점(Proprietary) 라이선스**라서 **서비스 안에서 쓰는 것(플러그인 설치)** 외의 복사 · 수정 · 배포가 금지되어 있습니다.

7.7 기본 사용 — 자동 호출 · 직접 호출

방법 1
자동으로 — 요청만 하기

요청

"files/club-report.pdf 의 표를 엑셀로 옮겨줘"

동작

description을 보고 **pdf 스킬로 읽고 → xlsx 스킬로 만들기**를 스스로 이어서 사용

방법 2
직접 — 슬래시로 부르기

입력

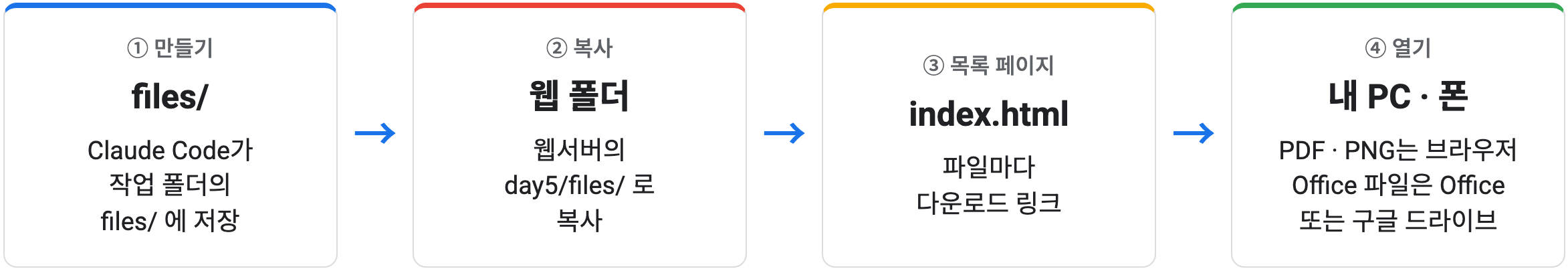
`/document-skills:pdf` notes/club-report.md 로 2쪽 PDF 만들어줘

동작

이름을 지정해 **확실하게** 그 스킬로 작업

좋은 요청에 담을 것	예시
입력 — 무엇을 바탕으로	`notes/club-report.md` · `files/club-report.pdf` · 내 사이트 `site/`
형식 · 분량	PDF 2쪽 · Word 1쪽 · PPT 3장 · 표 + 수식 + 차트
꼭 들어갈 것	요약 3줄 · 할 일 표 · 내 사이트 색

7.8 결과 파일 확인 — 웹 링크로 내려받아 내 PC에서



- 확인 주소: `http://<공인 IP>/day5/files/` — 서버에 화면(GUI)을 설치하지 않고도 결과를 확인합니다.
- **Office가 없다면**: 구글 드라이브에 업로드 → 더블클릭 → 구글 문서 · 시트 · 슬라이드로 열기
- ①~③은 CLAUDE.md에 **규칙 한 줄**로 넣어 두면 매번 자동으로 진행됩니다. (실습 Lab 06)

⚠ **웹 폴더의 파일은 주소만 알면 누구나 받을 수 있습니다.** 실명 · 연락처 · 학번은 넣지 않고 **가명 · 예시 정보**로 만듭니다.

🌐 Part 8. 다른 AI 에이전트 둘러보기 — OpenClaw · Hermes

터미널의 Claude Code 밖에는 어떤 에이전트가 있을까 (소개만 · 실습 없음)

8.1 AI 에이전트 지형도 — 세 가지 스타일

터미널 집중형
Claude Code

어디서
터미널 · VS Code (지금 쓰는 것)

강점
폴더 · 코드 · 서버 작업을 사람이 **지켜보며 승인**

메신저 상주형
OpenClaw

어디서
내 컴퓨터 · 서버에 **24시간** 켜 두고 텔레그램 · 슬랙 · 디스코드 등 **메신저로 대화**

강점
언제 어디서나 말 걸기 · 알림 · 반복 작업

자가 학습형
Hermes Agent

어디서
터미널 · 메신저 · 서버(VPS)

강점
대화를 **오래 기억**하고, 복잡한 일을 마치면 **스스로 스킬을 만들어** 점점 나아짐

💡 세 가지 모두 **"스킬"** 과 **"기억"** 개념을 가지고 있습니다. 오늘 배운 CLAUDE.md · SKILL.md는 다른 에이전트를 이해하는 **기본기**입니다.

8.2 한눈에 비교

항목	Claude Code	OpenClaw	Hermes Agent
만든 곳	Anthropic	OpenClaw 재단 (오픈소스 커뮤니티)	Nous Research
라이선스	Anthropic 이용 약관	MIT (오픈소스)	MIT (오픈소스)
주로 쓰는 곳	터미널 · IDE	메신저 20여 종 + 대시보드	터미널 + 메신저 (텔레그램 · 슬랙 등)
기억	CLAUDE.md · 자동 메모리	내 컴퓨터에 저장되는 기억	장기 기억 · 지난 대화 검색 · 사용자 모델링
스킬	SKILL.md (플러그인 · 폴더)	스킬 · 플러그인 (ClawHub에서 공유)	스스로 만들고 고치는 스킬 (Agent Skills 표준 호환)
모델	Claude	Claude · 다른 회사 모델 · 로컬 모델 선택	여러 모델 공급자 중 선택
실행 방식	사람이 실행 · 승인	백그라운드 상주(데몬)	상주 + 예약 작업(cron)

8.2 한눈에 비교 (확대 보기)

항목	Claude Code	OpenClaw	Hermes Agent
주로 쓰는 곳	터미널 · IDE	메신저 + 대시보드	터미널 + 메신저
기억	CLAUDE.md · 자동 메모리	내 컴퓨터에 저장	장기 기억 · 대화 검색
스킬	SKILL.md	ClawHub 공유	스스로 만들고 고침
실행 방식	사람이 실행 · 승인	백그라운드 상주	상주 + 예약 작업

8.3 써 보기 전에 알아야 할 것 — 계정 · 보안

주의 1
Claude 계정 연결

다른 에이전트에서 Claude를 쓰려면 **API 키**(사용량만큼 별도 요금)로 연결합니다. Claude **구독 계정 로그인 정보**를 다른 도구에 넣어 쓰는 것은 **약관상 허용되지 않습니다.** → 팀 계정 제재 위험

주의 2
24시간 · 메신저

메신저로 들어오는 메시지는 **믿을 수 없는 입력**입니다. 누군가 보낸 메시지가 에이전트에게 **엉뚱한 명령**을 시킬 수 있습니다.

주의 3
권한 · 노출

기본 설정에서는 도구가 **내 컴퓨터에서 직접** 실행됩니다. 인터넷에 열어 두기 전에 **격리(sandbox) · 접근 제한** 설정이 필요합니다.

❗ **그래서 수업에서는 실습하지 않습니다.** 개념만 비교하고, 관심 있는 학생은 수업이 끝난 뒤 **개인 PC · 개인 API 키**로, 각 프로젝트의 **보안 안내 문서를 먼저 읽고** 시도합니다.

8.4 이럴 땐 무엇을? — 선택 가이드

하고 싶은 일	어울리는 도구	이유
내 웹사이트 · 코드를 만들고 고치기	**Claude Code**	파일 · 명령을 **승인하며** 정확하게
과제 · 보고서 · 발표자료 파일 만들기	**Claude Code** + 문서 스킬	스킬로 형식 품질 확보
휴대폰 메신저로 일정 알림 · 간단한 질문	**OpenClaw** 류	24시간 상주 · 메신저 연결
오래 쓰면서 나에게 맞춰지는 비서	**Hermes** 류	장기 기억 · 스킬 자동 생성

💡 **공통 원리는 같습니다**: 어떤 에이전트든 ① **규칙(기억)** ② **스킬(절차)** ③ **권한 · 승인(안전)** 을 먼저 살펴보면 금방 익힐 수 있습니다. — Day 5 · Day 6에서 배우는 것이 바로 이 세 가지입니다.

🐙 Part 9. Git으로 공유 & 실습 미리보기

규칙과 스킬도 "코드처럼" 관리한다

9.1 무엇을 올리고 무엇을 빼나?

파일 · 폴더	Git	이유
`CLAUDE.md`	✅	팀 · 프로젝트 공통 규칙
`.claude/rules/`	✅	주제별 공통 규칙
`.claude/skills/` (Day 6 나만의 스킬)	✅	팀원도 같은 스킬 사용
`docs/` (`@` 가져오는 문서)	✅	CLAUDE.md가 참조하는 문서
`CLAUDE.local.md`	✕	나만의 설정 → `.gitignore`
`~/.claude/` (계정 폴더 · 설치한 플러그인)	✕	로그인 정보 · 개인 설정
`.pem` · 비밀번호 · API 키	🚫	**절대 금지**

💡 **팀 과제 · 팀 프로젝트에서**: 저장소를 받은 팀원은 `claude` 만 실행해도 **같은 규칙 · 같은 스킬**로 작업을 시작합니다.

9.2 실습 주제 — 하나를 골라 진행

주제 A
동아리 · 학과 소개 사이트

페이지

소개 · 활동 · 멤버 · 모집

규칙 포인트

색 · 글꼴 · 메뉴를 모든 페이지에 똑같이

주제 B
나만의 학습 도우미

페이지

과목별 정리 노트 · 퀴즈 · 용어 사전

규칙 포인트

노트 형식 · 퀴즈 형식 통일

주제 C
나의 포트폴리오

페이지

자기소개 · 프로젝트 · 수상 · 연락처

규칙 포인트

카드 디자인 · 문장 톤 통일

- 공통 결과물: `/var/www/html/day5/` 에 배포 → `http://<공인 IP>/day5/` 에서 확인
- 주제가 달라도 배우는 것은 같습니다: **규칙 없이 → 규칙 있게 → 나누고 → 교정 저장 → 문서 스킬**

9.3 Day 5 실습 진행 순서

Lab	내용	결과물	구분
00	실습 폴더 · 주제 · 전역 CLAUDE.md · **프로그램 설치(백그라운드)**	`/root/day5-lab/`	필수
01	CLAUDE.md **없이** 첫 페이지 만들기 · 관찰	`/day5/v0/`	필수
02	`/init` → 다듬기 → **같은 요청으로 다시** 만들어 비교	CLAUDE.md	필수
03	`@` 가져오기 · `.claude/rules/` 로 나누기 · `/context`	`docs/` · `rules/`	필수
04	교정 저장 · `CLAUDE.local.md`	규칙 추가	필수
05	**문서 스킬 설치** — 플러그인 (막히면 대안 A · B)	`/skills` 목록	필수
06	**PDF 읽고 바꾸기** — PDF 만들기 → PNG · Excel · Word · PPT 중 선택	`/day5/files/`	1종 필수
07	배포 · v0 비교 · (선택) Git · 서버 이미지 · FAQ	`yau-web-img-day5`	필수

✅ 셀프 체크 퀴즈

1. CLAUDE.md를 **짧게** 써야 하는 이유는? (컨텍스트 창과 관련지어)
2. `CLAUDE.md` 와 `CLAUDE.local.md` 중 **Git에 올리지 않는** 파일은?
3. `.claude/rules/html.md` 맨 위에 `paths: "**/*.html"` 을 쓰면 언제 읽히나요?
4. 같은 지적을 **두 번째** 하게 되었을 때 할 일은?
5. `/clear` 를 해도 **남는 것**은 대화 내용과 CLAUDE.md 중 무엇인가요?
6. 플러그인으로 설치한 Word 스킬을 **직접** 부르는 명령은?
7. 메신저로 대화하며 24시간 동작하는 에이전트 · 스스로 스킬을 만드는 에이전트는 각각?

> **정답**: ① 매 대화마다 항상 읽혀 자리를 차지하고, 길수록 중요한 규칙이 묻힘 ② `CLAUDE.local.md` ③ HTML 파일을 읽거나 고칠 때 ④ "방금 고친 내용을 CLAUDE.md에 한 줄로 추가해줘" ⑤ CLAUDE.md ⑥ `/document-skills:docx` ⑦ OpenClaw · Hermes Agent

Q & A

수고하셨습니다! 잠시 휴식 후 핸즈온 실습을 시작합니다.

강사: 신인철

소속: 클라우드스퀘어